Time Table & Trigger Time Correction

April 23, 2026

Outline

1. Related DocDB Entries 3
2. `pueo::RawHeader` Post-Processing 5
3. Time Table 7

Outline

Outline 1
1. Related DocDB Entries 3
2. `pueo::RawHeader` Post-Processing 5
3. Time Table 7
   - 3.1. File Tree 8
   - 3.2. Bad Runs: Weird pps (pre-amp runs) 11
   - 3.3. Bad Runs: Too Short! 12
   - 3.4. Bad Runs: No Timestamp found 13
   - 3.5. Bad Runs: Year 1970, but recoverable! 15
   - 3.6. Bad Runs: Missing Seconds, recoverable! 16
   - 3.7. `corrected_event_second` Sanity Checks 17

1. Related DocDB Entries

1. Patrick's notes *Event Time to Real Time in PUEO* (**doc-588-v1**):
   https://pueo.uchicago.edu/DocDB/cgi-bin/ShowDocument?docid=588
   - Mainly this talks about `last_pps` correction via interpolation/extrapolation (and a sliding average)
   - ie smoothing out the 8-ns SYSCLK quantization error and fixing occasional pps glitches (glitches only happened in pre-amp runs where `run<783`)
2. Jason's collaboration meeting slides on global timing calibration (**doc-607-v5**):
   https://pueo.uchicago.edu/DocDB/cgi-bin/ShowDocument?docid=607
   - This also includes `event_second` correction which can sometimes reset back to year 1970.
   - Also included: correction procedures, related Slack Q&A, and some lists of "bad runs"

2. `pueo::RawHeader` Post-Processing

- For runs that can be post-processed, `pueoEvent/macros/trigger_time_correction.C` is used to expand the original header.
  - For these runs, `pueo::RawHeader.corrected_trigger_time` is populated.
  - This is of type `TTimeStamp`. To get the second, do `corrected_trigger_time.GetSec()`
  - **To get the subsecond, call `corrected_trigger_time.GetNanoSec()`**
- For runs that cannot be post-processed, the macro is basically a no-opt and only copies the original header to make a new header.
  - For these runs, `corrected_trigger_time` is simply **initialized to zero** (year 1970).
- Some simple sanity checks have been carried out to make sure that the copying works. That is, fields that are supposed to be the same stay the same...

3. Time Table

3.1. File Tree

- Once you untar the tarball `time_tables.tar.gz`, you should see a folder `time_tables/` which simply mimics the structure of `$PUEO_ROOT_DATA`
- There is also an `error_log/` folder:

```
error_log/
├── 1000.err
├── 1000.out
├── 1001.err
├── 1001.out
├── 1002.err
├── 1002.out
```

- Time tables made by `pueoEvent/macros/make_time_tables.C`

```
time_tables/
├── run1000
│   ├── time_table.root
│   ├── time_table.txt
│   └── time_table_color.txt
├── run1001
│   ├── time_table.root
│   ├── time_table.txt
│   └── time_table_color.txt
├── run1002
│   ├── time_table.root
│   ├── time_table.txt
│   └── time_table_color.txt
├── run1003
│   ├── time_table.root
│   ├── time_table.txt
│   └── time_table_color.txt
```

3.1. File Tree

- `time_table.txt` and `time_table_color.txt` are the same,
  The former is meant to be read by a csv reader, the latter is meant to be printed to terminal:

```
└─>$ cat time_tables/run889/time_table_color.txt

Color: Has GPS Timestamp  Anchor Row  Invalid Delta (Inserted) Missing Second and Invalid Delta
Anchor row is where corrected_this_pps equals this_pps exactly.
Relative delta is defined to be (next_pps - this_pps) - 125000000
```

run number	event_second from DAQ	corrected event_second	readout time (sec)	this_pps	next_pps	relative delta	avg. rel. delta	corrected this_pps
uint32_t	int32_t	int32_t	int32_t	uint32_t	uint32_t	int32_t	double	double
889	1766554874	1766554874	1766554876	0	0	0	34.88	4273811011.71
889	1766554875	1766554875	1766554876	103843750	228843785	35	34.88	103843750.59
889	1766554876	1766554876	1766554877	228843785	353843820	35	34.88	228843785.47
889	1766554877	1766554877	1766554878	353843820	478843855	35	34.88	353843820.35
889	1766554878	1766554878	1766554879	478843855	603843889	34	34.88	478843855.24
889	1766554879	1766554879	1766554880	603843889	728843925	36	34.88	603843890.12
889	1766554880	1766554880	1766554881	728843925	853843959	34	34.88	728843925.00
889	1766554881	1766554881	1766554882	853843959	978843995	36	34.88	853843959.88
889	1766554882	1766554882	1766554883	978843995	1103844029	34	34.88	978843994.76
889	1766554883	1766554883	1766554884	1103844029	1228844064	35	34.88	1103844029.65
889	1766554884	1766554884	1766554885	1228844064	1353844100	36	34.94	1228844064.53
889	1766554885	1766554885	1766554886	1353844100	1478844134	34	34.94	1353844099.47
889	1766554886	1766554886	1766554887	1478844134	1603844170	36	34.94	1478844134.41
889	1766554887	1766554887	1766554888	1603844170	1728844204	34	34.94	1603844169.35
889	1766554888	1766554888	1766554889	1728844204	1853844239	35	34.94	1728844204.29
889	1766554889	1766554889	1766554890	1853844239	1978844274	35	34.94	1853844239.24
889	1766554890	1766554890	1766554891	1978844274	2103844309	35	34.94	1978844274.18
889	1766554891	1766554891	1766554892	2103844309	2228844343	34	34.94	2103844309.12
889	1766554892	1766554892	1766554893	2228844343	2353844379	36	34.94	2228844344.06
889	1766554893	1766554893	1766554894	2353844379	0	0	34.94	2353844379.00

3.1. File Tree

- And this is how you would read `time_table.root`, which contains the same thing as the text files:
  In the ROOT REPL, type

```
TFile f("time_tables/run889/time_table.root");
TTree * t = f.Get<TTree>("time_table_tree");
// show one (the first) entry
t->Show(0);

// can also print all columns, many entries
// t->Scan("*");
```

- Example result:

```
root [2] t→Show(0)
======> EVENT:0
 run                = 889
 event_second       = 1766554874
 corrected_event_second = 1766554874
 readout_time_sec = 1766554876
 this_pps           = 0
 next_pps           = 0
 relative_delta     = 0
 avg_relative_delta = 34.8824
 corrected_pps      = 4.27381e+09
 missing            = 0
 invalid_delta      = 1
 is_anchor_row      = 0
 has_timestamp      = 1
root [3]
```

3.2. Bad Runs: Weird pps (pre-amp runs)

- In some runs, if you take `next_pps` - `this_pps` you'd sometimes get crazy large numbers that deviate from the nominal clock frequency (125MHz) by up to ~ 1 KHz.
- This means that for some `event_seconds`, if we use the `pps` for subsecond calculations, we would get a wrong value.
- These only happened in pre-amp runs (before `783`), so they are ignored.
- That is, the folders `time_tables/run<bad#1>/` of these runs would be empty and **there will be no post-processing**

```
└─>$ cat error_log/*.err | grep "pre-amp"
Note: run 755 is a pre-amp run and will be ignored.
Note: run 756 is a pre-amp run and will be ignored.
Note: run 757 is a pre-amp run and will be ignored.
Note: run 758 is a pre-amp run and will be ignored.
Note: run 759 is a pre-amp run and will be ignored.
Note: run 760 is a pre-amp run and will be ignored.
Note: run 761 is a pre-amp run and will be ignored.
Note: run 762 is a pre-amp run and will be ignored.
Note: run 763 is a pre-amp run and will be ignored.
Note: run 764 is a pre-amp run and will be ignored.
Note: run 765 is a pre-amp run and will be ignored.
Note: run 766 is a pre-amp run and will be ignored.
Note: run 767 is a pre-amp run and will be ignored.
Note: run 768 is a pre-amp run and will be ignored.
Note: run 769 is a pre-amp run and will be ignored.
Note: run 770 is a pre-amp run and will be ignored.
Note: run 771 is a pre-amp run and will be ignored.
Note: run 772 is a pre-amp run and will be ignored.
Note: run 773 is a pre-amp run and will be ignored.
Note: run 774 is a pre-amp run and will be ignored.
Note: run 775 is a pre-amp run and will be ignored.
Note: run 776 is a pre-amp run and will be ignored.
Note: run 777 is a pre-amp run and will be ignored.
Note: run 778 is a pre-amp run and will be ignored.
Note: run 779 is a pre-amp run and will be ignored.
Note: run 780 is a pre-amp run and will be ignored.
Note: run 781 is a pre-amp run and will be ignored.
Note: run 782 is a pre-amp run and will be ignored.
```

3.3. Bad Runs: Too Short!

- `pps` correction is impossible without at least a valid `pps`
- The first and last second of any run have invalid pps:

```
└─>$ cat error_log/1020.err

Color: Has GPS Timestamp  Anchor Row  Invalid Delta (Inserted) Missing Second and Invalid Delta
Anchor row is where corrected_this_pps equals this_pps exactly.
Relative delta is defined to be (next_pps - this_pps) - 125000000
-------------------------------------------------------------------------------------------------------------------
run      | event_second | corrected    | readout     | this_pps  | next_pps  | relative | avg. rel. | corrected
number   | from DAQ     | event_second | time (sec)  |           |           | delta    | delta     | this_pps
uint32_t | int32_t      | int32_t      | int32_t     | uint32_t  | uint32_t  | int32_t  | double    | double
-------------------------------------------------------------------------------------------------------------------
 1020      1766944440     0              1766944420    1614510102  0           0          0.00                0.00
 -------------------------------------------------------------------------------------------------------------------
Fatal Error: time table too short (run 1020).
```

- So for runs shorter than 3 seconds, `time_tables/run<bad#2>/` **would be empty**. And there will be **no post-processing**. In the error logs we can dig these runs out:

```
└─>$ cat error_log/*.err | grep "too short"
Fatal Error: time table too short (run 1020).
Fatal Error: time table too short (run 1029).
Fatal Error: time table too short (run 1108).
Fatal Error: time table too short (run 1122).
Fatal Error: time table too short (run 1138).
Fatal Error: time table too short (run 1180).
Fatal Error: time table too short (run 1222).
Fatal Error: time table too short (run 1286).
Fatal Error: time table too short (run 1315).
Fatal Error: time table too short (run 1345).
Fatal Error: time table too short (run 1365).
```

3.4. Bad Runs: No Timestamp found

- To correct the `event_second` (aka `RawHeader.TriggerTime`) we need at least one event in the run to be timestamped (ie has a corresponding `timemarks_t` packet, which usually gets sent ~ every 100 events)
- Some early runs ($\leq$ 793) came without GPS timestamps
- We can still process the `last_pps` of these runs, **but not the `event_second`**.
- The folders `time_tables/run<bad#2>/` would be populated, although less useful

```
└─>$ cat time_tables/run785/time_table_color.txt

Color: Has GPS Timestamp  Anchor Row  Invalid Delta (Inserted) Missing Second and Invalid Delta
Anchor row is where corrected_this_pps equals this_pps exactly.
Relative delta is defined to be (next_pps - this_pps) - 125000000
```

run number	event_second from DAQ	corrected event_second	readout time (sec)	this_pps	next_pps	relative delta	avg. rel. delta	corrected this_pps
uint32_t	int32_t	int32_t	int32_t	uint32_t	uint32_t	int32_t	double	double
785	1766189287	0	1766189287	0	0	0	22.18	4265156254.53
785	1766189288	0	1766189287	95188981	220189003	22	22.18	95188980.71
785	1766189289	0	1766189288	220189003	345189025	22	22.18	220189002.88
785	1766189290	0	1766189289	345189025	470189047	22	22.18	345189025.06
785	1766189291	0	1766189290	470189047	595189069	22	22.18	470189047.24
785	1766189292	0	1766189291	595189069	720189091	22	22.18	595189069.41
785	1766189293	0	1766189292	720189091	845189114	23	22.18	720189091.59
785	1766189294	0	1766189293	845189114	970189136	22	22.18	845189113.76
785	1766189295	0	1766189294	970189136	1095189159	23	22.18	970189135.94
785	1766189296	0	1766189295	1095189159	1220189181	22	22.18	1095189158.12
785	1766189297	0	1766189296	1220189181	1345189203	22	22.18	1220189180.29
785	1766189298	0	1766189297	1345189203	1470189225	22	22.18	1345189202.47
785	1766189299	0	1766189298	1470189225	1595189247	22	22.18	1470189224.65
785	1766189300	0	1766189299	1595189247	1720189269	22	22.18	1595189246.82
785	1766189301	0	1766189300	1720189269	1845189291	22	22.18	1720189269.00
785	1766189302	0	1766189301	1845189291	1970189313	22	22.12	1845189291.18
785	1766189303	0	1766189302	1970189313	2095189335	22	22.12	1970189313.29
785	1766189304	0	1766189303	2095189335	2220189358	23	22.06	2095189335.41
785	1766189305	0	1766189304	2220189358	2345189380	22	22.12	2220189357.47
785	1766189306	0	1766189305	2345189380	2470189402	22	22.12	2345189379.59
785	1766189307	0	1766189306	2470189402	2595189424	22	22.12	2470189401.71
785	1766189308	0	1766189307	2595189424	2720189446	22	22.12	2595189423.82
785	1766189309	0	1766189308	2720189446	2845189468	22	22.12	2720189445.94

Figure 1: example time table for a run without any GPS timestamp

3.4. Bad Runs: No Timestamp found

- Using the error logs we can list 'em

```
└─>$ cat error_log/*.err | grep "failed"
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 785).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 786).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 787).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 788).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 789).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 790).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 791).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 792).
Fatal Error: The `event_second` seem okay, but none of them is corrected because I have failed to find a single timestamped event in this run (run 793).
```

- As discussed on Monday Apr 20, 2026, There will be **no post-processing** for the `pueo::RawHeader.triggerTime` in this case:
  - That is, `RawHeader.corrected_trigger_time.GetSec()` will **simply give you zero** (ie you'd be in year 1970)
  - For these runs, `RawHeader.triggerTime` are mostly okay in the sense that they are not clearly wrong, although we cannot guarantee their absolute correctness at the moment.

    So, if you really need to use these runs, **use `triggerTime`** instead of `corrected_triggerTime`

3.5. Bad Runs: Year 1970, but recoverable!

- Some runs have their `event_seconds` reset back to the Unix epoch.

```
└─>$ cat error_log/*.err | grep "launch second"
Warning: `event_second` < launch second 1766163240 (run 865).
Warning: `event_second` < launch second 1766163240 (run 1023).
Warning: `event_second` < launch second 1766163240 (run 1048).
Warning: `event_second` < launch second 1766163240 (run 1080).
Warning: `event_second` < launch second 1766163240 (run 1102).
Warning: `event_second` < launch second 1766163240 (run 1103).
Warning: `event_second` < launch second 1766163240 (run 1134).
Warning: `event_second` < launch second 1766163240 (run 1159).
```

- Fortunately, these runs all came with (many!) GPS timestamps which we can use to correct the `event_second`
- In this case, post-processing for `RawHeader.triggerTime` will take place, so you could safely call the `corrected_trigger_time`.

```
└─>$ cat time_tables/run865/time_table_color.txt

Color: Has GPS Timestamp  Anchor Row  Invalid Delta (Inserted) Missing Second and Invalid Delta
Anchor row is where corrected_this_pps equals this_pps exactly.
Relative delta is defined to be (next_pps - this_pps) - 125000000
```

run number | uint32_t	event_second from DAQ | int32_t	corrected event_second | int32_t	readout time (sec) | int32_t	this_pps | uint32_t	next_pps | uint32_t	relative delta | int32_t	avg. rel. delta | double	corrected this_pps | double
865	1314	1766478374	1766478374	0	0	0	33.29	4269988345.47
865	1315	1766478375	1766478379	100021083	225021116	33	33.29	100021082.76
865	1316	1766478376	1766478381	225021116	350021150	34	33.29	225021116.06
865	1317	1766478377	1766478381	350021150	475021182	32	33.29	350021149.35
865	1318	1766478378	1766478381	475021182	600021216	34	33.29	475021182.65
865	1319	1766478379	1766478382	600021216	725021249	33	33.29	600021215.94
865	1320	1766478380	1766478382	725021249	850021282	33	33.29	725021249.24
865	1321	1766478381	1766478382	850021282	975021316	34	33.29	850021282.53
865	1322	1766478382	1766478383	975021316	1100021349	33	33.29	975021315.82
865	1323	1766478383	1766478383	1100021349	1225021383	34	33.29	1100021349.12
865	1324	1766478384	1766478384	1225021383	1350021416	33	33.35	1225021382.41
865	1325	1766478385	1766478385	1350021416	1475021449	33	33.24	1350021415.76
865	1326	1766478386	1766478386	1475021449	1600021482	33	33.35	1475021449.00
865	1327	1766478387	1766478387	1600021482	1725021516	34	33.29	1600021482.35
865	1328	1766478388	1766478388	1725021516	1850021549	33	33.29	1725021515.65
865	1329	1766478389	1766478389	1850021549	1975021582	33	33.35	1850021548.94
865	1330	1766478390	1766478390	1975021582	2100021616	34	33.29	1975021582.29
865	1331	1766478391	1766478391	2100021616	2225021649	33	33.35	2100021615.59
865	1332	1766478392	1766478392	2225021649	2350021683	34	33.29	2225021648.94
865	1333	1766478393	1766478393	2350021683	2475021715	32	33.29	2350021682.24
865	1334	1766478394	1766478394	2475021715	2600021749	34	33.29	2475021715.53
865	1335	1766478395	1766478395	2600021749	2725021782	33	33.35	2600021748.82
865	1336	1766478396	1766478396	2725021782	2850021815	33	33.29	2725021782.18
865	1337	1766478397	1766478397	2850021815	2975021849	34	33.29	2850021815.47
865	1338	1766478398	1766478398	2975021849	3100021882	33	33.35	2975021848.76

3.6. Bad Runs: Missing Seconds, recoverable!

- Some runs are missing a few seconds in the middle.

```
└─>$ cat error_log/*.err | grep "missing"
Warning: missing seconds inserted (run 840).
Warning: missing seconds inserted (run 1230).
Warning: missing seconds inserted (run 1281).
Warning: missing seconds inserted (run 1282).
Warning: missing seconds inserted (run 1311).
Warning: missing seconds inserted (run 1332).
Warning: missing seconds inserted (run 1393).
```

- Not that we care about these seconds because no data would have been taken during these periods
- Still, the time table would recover these `event_second`s and compute their `pps` through interpolation

840	1766398748	1766398748	1766398747	3260074715	3385074754	39	38.76	3260074715.34
840	1766398749	1766398749	1766398748	3385074754	3510074792	38	38.76	3385074754.10
840	1766398750	1766398750	1766398749	3510074792	3635074831	39	38.76	3510074792.87
840	1766398751	1766398751	1766398750	3635074831	3760074869	38	38.76	3635074831.63
840	1766398752	1766398752	1766398751	3760074869	3885074908	39	38.76	3760074870.40
840	1766398753	1766398753	1766398752	3885074908	4010074947	39	38.82	3885074909.16
840	1766398754	1766398754	1766398753	4010074947	4135074985	38	38.76	4010074947.99
840	1766398755	1766398755	1766398754	4135074985	4260075024	39	38.76	4135074986.75
840	1766398756	1766398756	1766398755	4260075024	90107767	39	38.76	4260075025.51
840	1766398757	1766398757	1766398756	90107767	0	0	38.76	90107768.28
0	1766398758	1766398758	0	0	0	0	38.76	215107807.04
840	1766398759	1766398759	1766398759	340107844	465107883	39	38.76	340107845.81
840	1766398760	1766398760	1766398759	465107883	590107922	39	38.82	465107884.57
840	1766398761	1766398761	1766398760	590107922	715107961	39	38.76	590107923.40
840	1766398762	1766398762	1766398761	715107961	840108000	39	38.76	715107962.16
840	1766398763	1766398763	1766398762	840108000	965108039	39	38.76	840108000.93
840	1766398764	1766398764	1766398763	965108039	1090108077	38	38.76	965108039.69
840	1766398765	1766398765	1766398764	1090108077	1215108116	39	38.82	1090108078.46
840	1766398766	1766398766	1766398765	1215108116	1340108155	39	38.82	1215108117.28
840	1766398767	1766398767	1766398766	1340108155	1465108194	39	38.82	1340108156.10
840	1766398768	1766398768	1766398767	1465108194	1590108233	39	38.76	1465108194.93
840	1766398769	1766398769	1766398768	1590108233	1715108271	38	38.76	1590108233.69
840	1766398770	1766398770	1766398769	1715108271	1840108309	38	38.76	1715108272.46
840	1766398771	1766398771	1766398770	1840108309	1965108348	39	38.76	1840108311.22
840	1766398772	1766398772	1766398771	1965108348	2090108387	39	38.71	1965108349.99
840	1766398773	1766398773	1766398772	2090108387	0	0	38.74	2090108388.69
0	1766398774	1766398774	0	0	0	0	38.75	2215108427.43
0	1766398775	1766398775	0	0	0	0	38.76	2340108466.18
0	1766398776	1766398776	0	0	0	0	38.76	2465108504.93
0	1766398777	1766398777	0	0	0	0	38.76	2590108543.69
0	1766398778	1766398778	0	0	0	0	38.76	2715108582.46
0	1766398779	1766398779	0	0	0	0	38.76	2840108621.22
0	1766398780	1766398780	0	0	0	0	38.76	2965108659.99
0	1766398781	1766398781	0	0	0	0	38.76	3090108698.75
0	1766398782	1766398782	0	0	0	0	38.76	3215108737.51
0	1766398783	1766398783	0	0	0	0	38.76	3340108776.28
0	1766398784	1766398784	0	0	0	0	38.76	3465108815.04
0	1766398785	1766398785	0	0	0	0	38.76	3590108853.81
0	1766398786	1766398786	0	0	0	0	38.76	3715108892.57
0	1766398787	1766398787	0	0	0	0	38.76	3840108931.34
0	1766398788	1766398788	0	0	0	0	38.76	3965108970.10
840	1766398789	1766398789	1766398789	4090109007	0	0	38.76	4090109008.87

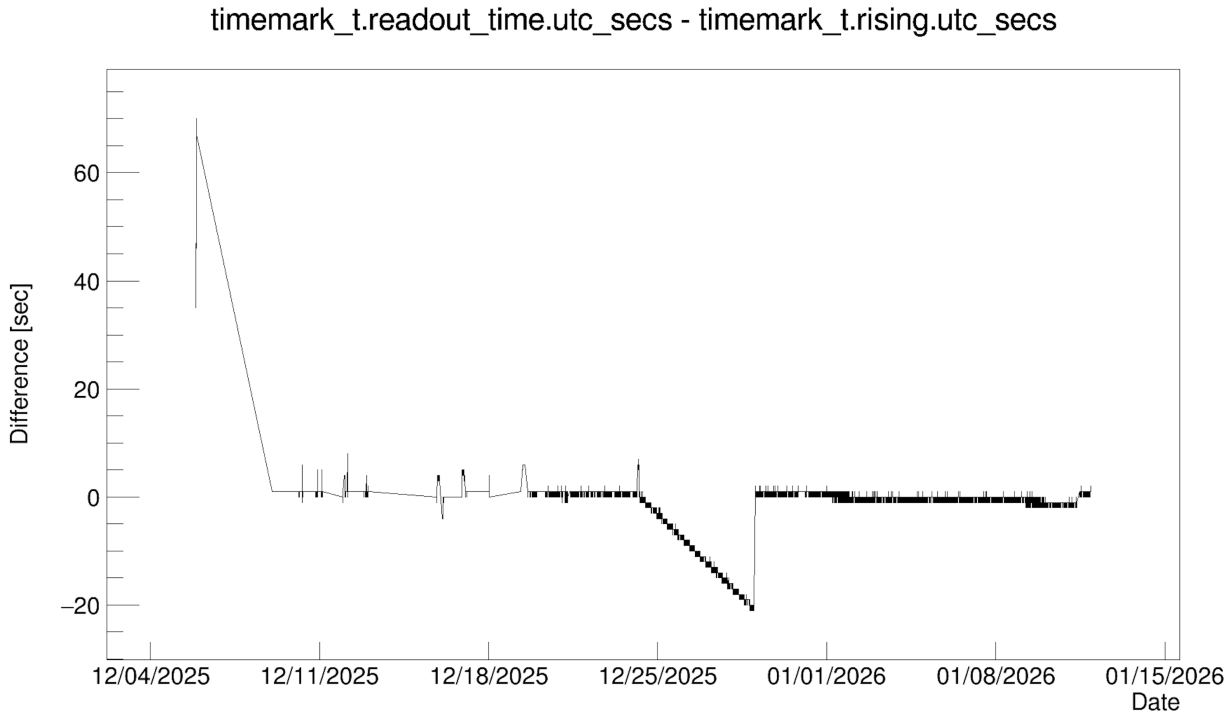
3.7. `corrected_event_second` Sanity Checks

Some simple sanity checks:

- Independently reconstructed `corrected_event_second`s are contiguous.
- Also, each `corrected_event_second` is equal to the orignal `event_second`
  - except when `event_second` goes clearly nuts
  - or when `corrected_event_second` cannot be determined because there's no GPS timestamp in the run
- Actually this is not always true, because the reconstruction is somewhat stochastic and imperfect, I cheated by throwing out accidentals (ie reconstructions that disagree with the original `event_second`)
- Pinky promise that accidentals don't happen often – the errors are logged and you can check.

3.7. `corrected_event_second` Sanity Checks

- Recall from the collaboration meeting that there was a Christmas ~~miracle~~ clock drift
- This plot shows the difference between the GPS timestamps and the computer time:



- We should therefore expect this trend in the `corrected_event_second` (which comes from the GPS timestamps).

3.7. `corrected_event_second` Sanity Checks

